

OPERATING SYSTEMS PRACTICAL

1) Program 1: One Process (Before fork())

```
Open  ▾  +  before_fork.c
~/OSLab/Fork

1 #include <stdio.h>
2 #include <unistd.h>
3 int main()
4 {
5     printf("PID = %d\n", getpid());
6     return 0;
7 }
```

Output :

```
rasagnya_07@LAPTOP-7LMHP1NN:~/OSLab/Fork$ gcc before_fork.c -o before_fork
rasagnya_07@LAPTOP-7LMHP1NN:~/OSLab/Fork$ ./before_fork
PID = 2735
```

2) Program 2: fork() Creates a Child

```
Open  ▾  +  fork.c
~/OSLab/Fork  Save  ≡  x

1 #include <stdio.h>
2 #include <unistd.h>
3 int main(){
4     fork();
5     printf("Hello\n");
6     return 0;
7 }
```

Output :

```
rasagnya_07@LAPTOP-7LMHP1NN:~/OSLab/Fork$ ./fork
Hello
Hello
```

3) Program 3: Parent vs Child

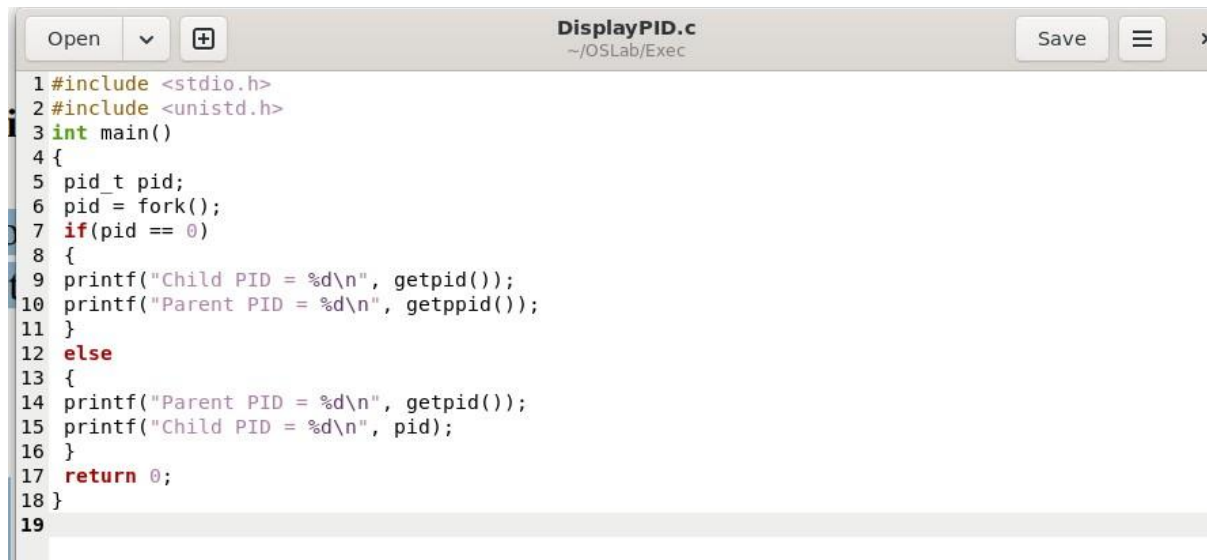
```
Open  ▾  +  ParentVsChild.c
~/OSLab/Fork  Save  ≡  x

1 #include <stdio.h>
2 #include <unistd.h>
3 int main(){
4     pid_t pid;
5     pid=fork();
6     if(pid==0){
7         printf("I am Child\n");
8     }else{
9         printf("I am Parent\n");
10    }
11 }
```

Output :

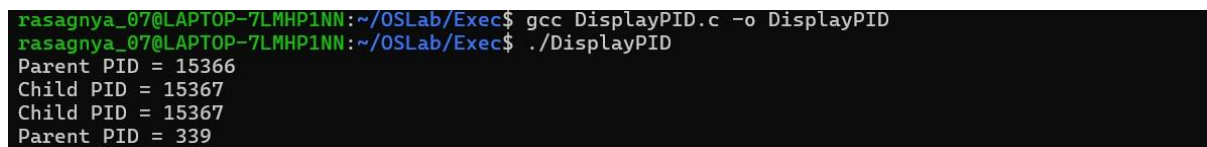
```
rasagnya_07@LAPTOP-7LMHP1NN:~/OSLab/Fork$ ./ParentVsChild
I am Parent
I am Child
```

4) Program 4: Display Both PIDs

A screenshot of a code editor window titled "DisplayPID.c" with the path "~/OSLab/Exec". The editor contains the following C code:

```
1 #include <stdio.h>
2 #include <unistd.h>
3 int main()
4 {
5     pid_t pid;
6     pid = fork();
7     if(pid == 0)
8     {
9         printf("Child PID = %d\n", getpid());
10        printf("Parent PID = %d\n", getppid());
11    }
12    else
13    {
14        printf("Parent PID = %d\n", getpid());
15        printf("Child PID = %d\n", pid);
16    }
17    return 0;
18 }
19
```

Output :

A screenshot of a terminal window showing the execution of the DisplayPID.c program. The commands and output are as follows:

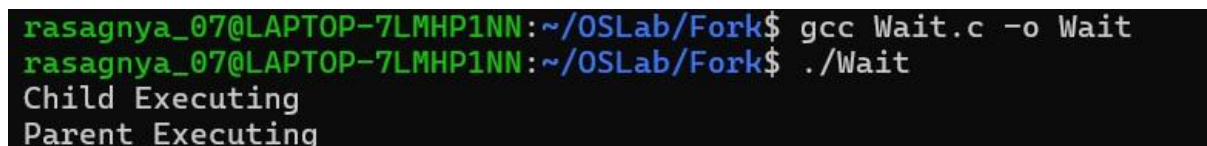
```
rasagnya_07@LAPTOP-7LMHP1NN:~/OSLab/Exec$ gcc DisplayPID.c -o DisplayPID
rasagnya_07@LAPTOP-7LMHP1NN:~/OSLab/Exec$ ./DisplayPID
Parent PID = 15366
Child PID = 15367
Child PID = 15367
Parent PID = 339
```

5) Program 5: Parent Waits for Child

A screenshot of a code editor window titled "Wait.c" with the path "~/OSLab/Fork". The editor contains the following C code:

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <sys/wait.h>
4 int main()
5 {
6     pid_t pid=fork();
7     if(pid==0){
8         printf("Child Executing\n");
9     }else{
10        wait(NULL);
11        printf("Parent Executing\n");
12    }
13    return 0;
14 }
```

Output :

A screenshot of a terminal window showing the execution of the Wait.c program. The commands and output are as follows:

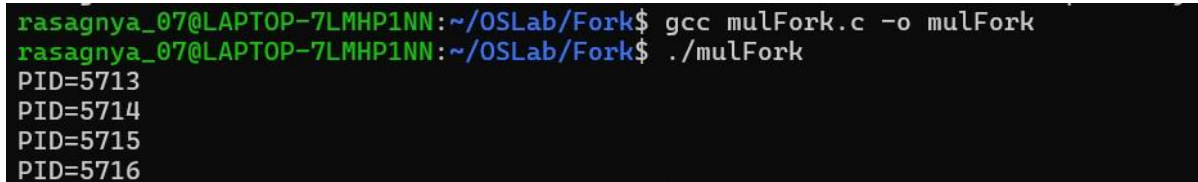
```
rasagnya_07@LAPTOP-7LMHP1NN:~/OSLab/Fork$ gcc Wait.c -o Wait
rasagnya_07@LAPTOP-7LMHP1NN:~/OSLab/Fork$ ./Wait
Child Executing
Parent Executing
```

6) Program 6: Multiple fork() Calls



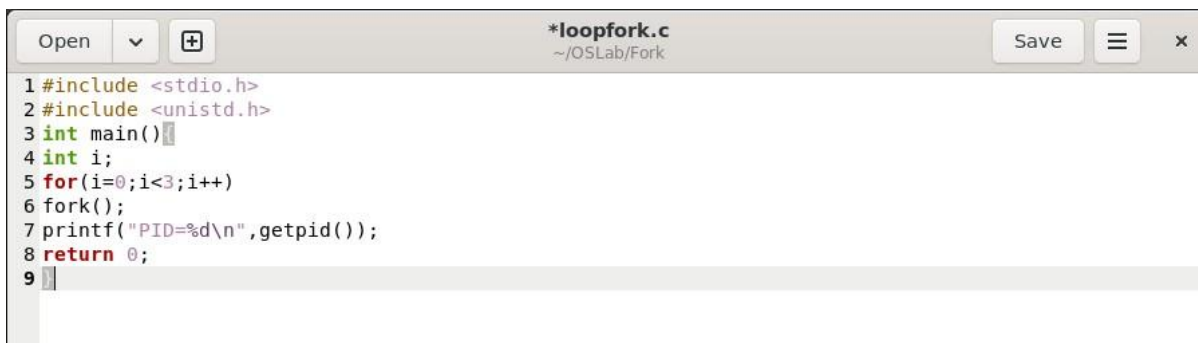
```
1 #include <stdio.h>
2 #include <unistd.h>
3 int main(){
4     fork();
5     fork();
6     printf("PID=%d\n", getpid());
7     return 0;
8 }
```

Output :



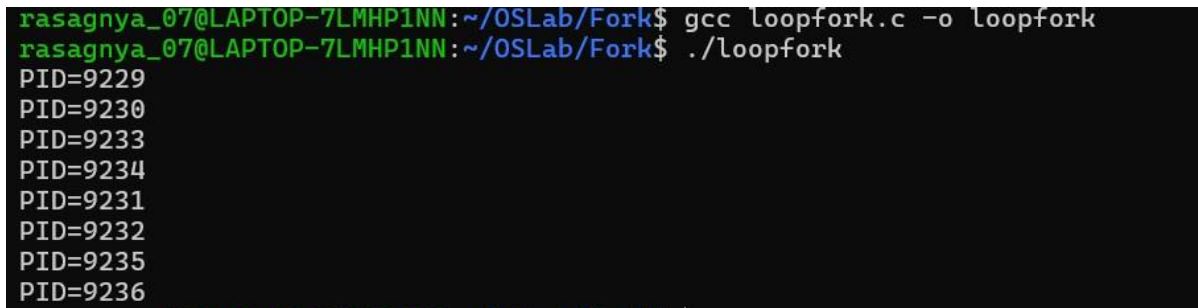
```
rasagnya_07@LAPTOP-7LMHP1NN:~/OSLab/Fork$ gcc mulFork.c -o mulFork
rasagnya_07@LAPTOP-7LMHP1NN:~/OSLab/Fork$ ./mulFork
PID=5713
PID=5714
PID=5715
PID=5716
```

7) Program 10: fork() Inside a Loop (Fun)



```
1 #include <stdio.h>
2 #include <unistd.h>
3 int main(){
4     int i;
5     for(i=0; i<3; i++)
6         fork();
7     printf("PID=%d\n", getpid());
8     return 0;
9 }
```

Output :



```
rasagnya_07@LAPTOP-7LMHP1NN:~/OSLab/Fork$ gcc loopfork.c -o loopfork
rasagnya_07@LAPTOP-7LMHP1NN:~/OSLab/Fork$ ./loopfork
PID=9229
PID=9230
PID=9233
PID=9234
PID=9231
PID=9232
PID=9235
PID=9236
```

8) Program 7: Child Executes Another Program (exec())

```
Open  ▾  +  exec.c  ~~/OSLab/Fork  Save  ≡  x

1 #include <stdio.h>
2 #include <unistd.h>
3 int main()
4 {
5     pid_t pid = fork();
6     if(pid == 0)
7     {
8         printf("Child executing ls...\n");
9         execl("/bin/ls", "ls", "-l", NULL);
10    }
11    else
12    {
13        printf("Parent Waiting...\n");
14    }
15    return 0;
16 }
```

Output :

```
rasagnya_07@LAPTOP-7LMHP1NN:~/OSLab/Fork$ gcc exec.c -o exec
rasagnya_07@LAPTOP-7LMHP1NN:~/OSLab/Fork$ ./exec
Parent Waiting...
Child executing ls...
rasagnya_07@LAPTOP-7LMHP1NN:~/OSLab/Fork$ total 160
-rwxr-xr-x 1 rasagnya_07 rasagnya_07 16088 Jul 29 03:16 DisplayPID
-rw-r--r-- 1 rasagnya_07 rasagnya_07 260 Jul 29 03:16 DisplayPID.c
-rwxr-xr-x 1 rasagnya_07 rasagnya_07 16000 Jul 29 03:12 ParentVsChild
-rw-r--r-- 1 rasagnya_07 rasagnya_07 146 Jul 29 03:11 ParentVsChild.c
-rwxr-xr-x 1 rasagnya_07 rasagnya_07 16032 Aug 1 03:15 Wait
-rw-r--r-- 1 rasagnya_07 rasagnya_07 195 Aug 1 03:15 Wait.c
-rwxr-xr-x 1 rasagnya_07 rasagnya_07 16000 Jul 31 09:00 before_fork
-rw-r--r-- 1 rasagnya_07 rasagnya_07 100 Jul 31 09:00 before_fork.c
-rwxr-xr-x 1 rasagnya_07 rasagnya_07 16032 Aug 1 03:36 exec
-rw-r--r-- 1 rasagnya_07 rasagnya_07 220 Aug 1 03:35 exec.c
-rwxr-xr-x 1 rasagnya_07 rasagnya_07 15992 Jul 29 03:08 fork
-rw-r--r-- 1 rasagnya_07 rasagnya_07 90 Jul 29 03:08 fork.c
-rwxr-xr-x 1 rasagnya_07 rasagnya_07 16040 Aug 1 03:34 loopfork
-rw-r--r-- 1 rasagnya_07 rasagnya_07 124 Aug 1 03:33 loopfork.c
-rwxr-xr-x 1 rasagnya_07 rasagnya_07 16040 Aug 1 03:21 mulFork
-rw-r--r-- 1 rasagnya_07 rasagnya_07 108 Aug 1 03:21 mulFork.c
```